

College Event Management System

Project Summary & Interview Notes

Generated: 07/02/2026

1. Project Overview

- A full-stack web app that manages college events and student registrations.
- Admins can create, update, delete events and view registered students.
- Students can register, log in, view events and enroll in them.

2. Tech Stack

- Frontend: React JS (SPA with React Router)
- Backend: Node.js + Express (REST API)
- Database: MySQL (relational data)
- Tools: Axios, dotenv, CORS

3. Architecture Flow

- React UI sends HTTP requests using Axios.
- Express API validates input and runs SQL queries.
- MySQL stores students, events, and registrations.

Example Flow: Student Registration

- User fills form on /register (React).
- POST /api/register sent to backend.
- Backend inserts student into MySQL.
- Success response returned to UI.

4. Core Features

Student

- Account registration and login
- View upcoming events
- Register for an event

Admin

- Create, update, delete events
- View all event registrations

General

- Environment-based API configuration
- Clear error handling and validations

5. Key API Endpoints

- POST /api/register
- POST /api/login
- POST /api/events/create
- GET /api/events/all
- PUT /api/events/update/:id
- DELETE /api/events/delete/:id
- POST /api/events/:eventId/register
- GET /api/events/:eventId/registrations

6. Database Schema (MySQL)

students

- id, username (unique), password, name, email, department, status

event

- id, title, description, date, time, location, registration_link

event_registrations

- event_id + student_id (unique), registered_at
- Foreign keys link to students and event tables

7. Deployment Notes

- Frontend uses REACT_APP_API_URL for API base URL.
- Backend uses .env for DB credentials and PORT.
- Build frontend with npm run build for deployment.

8. Interview Q&A (Short Answers)

Why this stack?

React is great for modular UI, Node/Express is fast for REST APIs, and MySQL fits relational data.

How does the system work end-to-end?

React sends API requests !' Express validates and queries !' MySQL stores data !' response updates UI.

How do you prevent duplicate registrations?

A unique key on (event_id, student_id) plus a pre-check in the API.

How would you secure passwords?

Use bcrypt to hash passwords and verify on login.

What would you improve?

Add JWT auth, role-based access, server-side validation library, and pagination.

How would you deploy?

Host React on Netlify/Vercel; backend on Render/Railway with env vars set.